

Clase 113 — Ataques del lado del cliente: CORS, postMessage y prototype pollution

Parte: 4 — Seguridad de aplicaciones web · Fuente: PortSwigger Research / Real-World Bug Hunting (Yaworski) 🕒 Duración estimada: 120 min · Nivel: Experto

🎯 Objetivo

Explotar tres clases de ataques **del lado del cliente** propias de las aplicaciones JavaScript modernas: configuraciones **CORS** inseguras que exponen datos, uso inseguro de **postMessage** entre ventanas/iframes, y **prototype pollution** en JavaScript que puede escalar a XSS o RCE (en Node). Son vectores de moda en el bug bounty actual.

⚠️ **Ética:** solo en labs propios/autorizados.

📖 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Auditar** políticas CORS y explotar reflejo del `origin` con credenciales.
- 2. **Detectar** manejadores `postMessage` sin validación de origen.
- 3. **Explotar** prototype pollution en cliente y (conceptualmente) en Node.
- 4. **Encadenar** prototype pollution hacia un "gadget" que produce XSS.
- 5. **Recomendar** allowlists de origen, validación de mensajes y `Object.freeze`.

📚 Temas

#	Tema	Por qué importa
1	Modelo de origen (SOP) y CORS	Base de la seguridad cliente
2	Configuraciones CORS inseguras	Fuga de datos con credenciales
3	postMessage inseguro	Comunicación entre ventanas
4	Prototype pollution: concepto	Contaminar <code>Object.prototype</code>
5	Gadgets y escalada a XSS/RCE	Impacto real
6	Herramientas (DOM Invader)	Detección práctica
7	Defensas por vector	Cierre del fallo

Definiciones y características

- **SOP (Same-Origin Policy)**: política que aísla orígenes distintos. Característica: CORS la relaja de forma controlada.
- **CORS inseguro**: reflejar el `Origin` o permitir `null` con `Access-Control-Allow-Credentials: true`. Característica: permite leer datos autenticados cross-origin.
- **postMessage**: API para comunicar ventanas/iframes. Característica: insegura si no se valida `event.origin` ni los datos.
- **Prototype pollution**: inyectar propiedades en `Object.prototype` vía claves como `__proto__`. Característica: afecta a todos los objetos.
- **Gadget**: código que lee una propiedad contaminable y la usa peligrosamente. Característica: convierte la contaminación en XSS/RCE.
- **Source/sink en cliente**: origen del dato y punto de uso. Característica: base para rastrear estos ataques.

Herramientas y preparación

- **PortSwigger labs** de CORS, DOM-based y prototype pollution.
- **Burp** con **DOM Invader** (detecta postMessage y prototype pollution).
- Un servidor propio para alojar la página de exploit cross-origin.

Laboratorio guiado

 Solo en labs propios.

1. **CORS**: envía una petición con `Origin: https://evil.com` y observa si se refleja en `Access-Control-Allow-Origin` junto a `Allow-Credentials: true`.
2. Aloja un exploit que use `fetch` con `credentials: 'include'` para leer datos sensibles cross-origin.
3. Prueba también el origen `null` (iframe sandbox) si el servidor lo acepta.
4. **postMessage**: con DOM Invader, localiza un handler que use `event.data` sin validar `event.origin`.
5. Envía un mensaje desde tu página para inyectar en un sink (p. ej. `innerHTML`) y lograr XSS.
6. **Prototype pollution**: identifica un source (parámetro JSON/URL) que permita `__proto__[x]=y` y un gadget que lo consuma.
7. Encadena la contaminación hasta ejecutar JavaScript en el lab. Documenta cada vector.

Ejercicios

1. Explota un CORS que refleja el Origin y roba datos autenticados en el lab.
2. Encuentra un postMessage sin validación de origen y explótalo.
3. Contamina `Object.prototype` desde un parámetro y comprueba el efecto global.
4. Localiza un gadget que convierta la contaminación en XSS.
5. Explica el riesgo de prototype pollution en Node (RCE) frente al cliente (XSS).
6. Propón defensas: allowlist de orígenes, validar `event.origin`, `Object.freeze(Object.prototype)`.

Reto verificable

Resuelve un lab de **CORS** que permita exfiltrar datos autenticados y un lab de **prototype pollution** que escale a XSS, documentando ambos. **Criterio de aceptación:** ambos labs quedan resueltos, entregas el exploit CORS (con `credentials`), el source→gadget de la contaminación y las defensas concretas por vector.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
CORS no refleja Origin	Allowlist estricta; no explotable por reflejo
<code>Allow-Credentials</code> ausente	Sin credenciales no hay fuga de datos privados
<code>postMessage</code> valida origen	Handler seguro; documenta la fortaleza
<code>__proto__</code> filtrado	La app sanea claves; prueba <code>constructor.prototype</code>
Contaminación sin gadget	No hay sink que la consuma; busca otro

? Preguntas frecuentes

? **¿CORS mal configurado es como CSRF?** No: CSRF ejecuta acciones; el CORS inseguro permite leer respuestas cross-origin, filtrando datos.

? **¿Prototype pollution siempre es explotable?** No por sí sola; necesita un gadget que lea la propiedad contaminada. Sin gadget, es un fallo latente.

? **¿Por qué validar `event.origin` en `postMessage`?** Porque sin ello cualquier página puede enviar mensajes maliciosos que tu handler procesa como confiables.

Referencias

- PortSwigger — CORS: <https://portswigger.net/web-security/cors>
- PortSwigger — Prototype pollution: <https://portswigger.net/web-security/prototype-pollution>
- MDN — Window.postMessage: <https://developer.mozilla.org/en-US/docs/Web/API/Window/postMessage>
- Yaworski, *Real-World Bug Hunting*.

Siguiente clase

Clase 114 - Bug bounty: metodología y plataformas